

U-Hermes

便携式 AI Agent · 使用文档

插上 U 盘就能用 · 无需安装 · 无残留

版本 0.1 · 2026-05

1. 快速开始

U-Hermes 是一个完整的 AI Agent，所有运行时（Python、依赖、配置）都装在 U 盘里。插上电脑就能用，拔掉就走，本地不留痕迹。

1.1 拿到 U 盘后做什么

```
Windows:    双击    Windows-Start.bat
macOS:      双击    Mac-Start.command (或在终端运行  bash Mac-Start.command)
```

就这一步。启动后访问 <http://127.0.0.1:8787/> 就能看到 Hermes 的 Web UI。关闭命令行窗口即可停止服务。

首次启动 vs 后续启动

- **首次启动**（U 盘是从仓库克隆的裸版本，还没有 `app/` 目录）：启动脚本会自动下载独立 Python 解释器、PortableGit 和所有 Python 依赖到 U 盘的 `portable/app/` 目录。需要联网，国内会自动用清华、华为云、npmmirror 等镜像。总共下载约 1 GB，耗时几分钟到十几分钟。
- **后续启动**：检测到 `app/` 已完备，直接跳过下载环节，秒开。

1.2 第一次启动会发生什么

- 自动绑定设备指纹，生成本机专属的 API key（保存在 `portable/data/.hermes/.env`）
 - 初始化运行时数据目录 `portable/data/.hermes/`
 - 启动 Web 服务，监听本地 `127.0.0.1:8787`（不对外开放）
 - 浏览器打开 UI，可以直接开始对话
-

2. U 盘里有什么

U 盘根目录的结构如下。你直接接触的只有最外层两个启动脚本，其它都是程序内部细节。

u-hermes/
├── Windows-Start.bat
├── Mac-Start.command
├── README.md
├── portable/
│ ├── webui/
│ ├── agent/
│ ├── app/
│ │ ├── runtime/
│ │ │ ├── python-win-x64/
│ │ │ │ └── git-win-x64/
│ │ └── packages/
├── data/.hermes/
└── workspace/

<-- U 盘根目录

Windows 双击启动（首次自动装依赖）

macOS 双击启动（首次自动装依赖）

项目说明

程序内部（不需要进入）

Web UI 源码

AI Agent 源码

首次启动下载的运行时

独立 Python

PortableGit（提供 bash/grep/git）

所有 pip 依赖

运行时配置、会话历史、API key

AI 写代码、改文件的工作区

2.1 你常用的目录

目录	说明
workspace/	AI 帮你写的代码、文件默认放这里。可以自己往里塞文件让 AI 处理。
data/.hermes/	配置文件 config.yaml、API key、对话历史。删了会重置成出厂状态。
data/logs/	运行日志，遇到问题时翻这里。

3. 跨电脑使用 U 盘

U-Hermes 设计成可以在多台电脑之间无缝迁移。但有几个需要知道的点：

3.1 盘符变化自动处理

U 盘在不同电脑上可能挂载到不同位置 (D:、F:、G:，或 `/Volumes/USB1`)。启动脚本会自动检测：如果发现路径变了，会把上次的运行时状态备份到 `data/.hermes.bak`，然后用全新的状态目录启动，避免缓存里的死路径引发问题。

会丢什么？ 切换电脑后，之前的对话历史在备份目录里但不会自动加载。如果想恢复某次对话，可以手动把 `data/.hermes.bak` 里的 `sessions` 文件挪回 `data/.hermes`。

3.2 Windows 不需要预装 Git

Hermes 的 shell 工具依赖 `bash`、`grep`、`find`、`git` 这类 Unix 工具。我们把 **PortableGit** 直接打包到了 U 盘里 (`portable/app/runtime/git-win-x64/`，首次启动自动下载)，所以 Windows 端不需要任何预装软件。启动脚本会自动设置 `HERMES_GIT_BASH_PATH`，让 agent 用 U 盘上的 `bash`。

为什么不用 WSL？ Windows 自带的 `bash.exe` 实际上是 WSL 的启动器。WSL 配置复杂、容易因代理问题坏掉。自带 PortableGit 简单稳定，不依赖外部安装。

3.3 macOS 不需要额外依赖

macOS 系统自带 `bash` 和一套 Unix 工具 (`grep`、`find`、`curl` 等)，启动脚本直接复用。

3.4 跨平台 U 盘

默认情况下，首次启动只会下载**当前系统架构**对应的 Python。如果希望同一只 U 盘既能在 Mac 用又能在 Windows 用，只需在另一个系统上也跑一次 `Start.*`，会自动补上该系统的 Python 运行时和依赖。

`portable/agent/`、`portable/webui/` 本身是纯 Python，跨平台共享。

4. 隐私 · 本机会留下什么

U-Hermes 设计目标之一就是「拔掉 U 盘后本地无痕」。具体来说：

4.1 完全留在 U 盘上

- Python 解释器、所有 pip 包：`portable/app/`
- API key、设备指纹：`portable/data/.hermes/.env`
- 对话历史、配置：`portable/data/.hermes/`
- AI 生成的代码、文件：`portable/workspace/`
- 日志：`portable/data/logs/`

4.2 可能留在本机的痕迹

痕迹	位置	说明
pip 缓存	<code>~/.cache/pip</code> 或 <code>%LOCALAPPDATA%\pip\Cache</code>	setup 时下载 wheel 会缓存。可手动删除。
Shell 历史	<code>~/.bash_history</code> 或 <code>~/.zsh_history</code>	你在终端里跑过的命令。
浏览器历史	浏览器本身	如果在 Hermes UI 里登录过外部服务，建议用隐私模式。

4.3 完全不会留下

- 不修改系统 PATH 或环境变量
- 不写注册表 (Windows)
- 不装系统级 Python 或任何依赖
- 不创建 launchd / 开机启动项
- 不留 venv 或 `~/.local` 痕迹

完全无痕方案：如果连 pip 缓存都不想留，在跑 setup 之前先 `export PIP_NO_CACHE_DIR=1` (Windows: `set PIP_NO_CACHE_DIR=1`)，这样下载的 wheel 不会缓存到本机。

5. 常见问题

5.1 启动后端口被占用

默认端口是 8787。如果被占了，Mac 启动器会自动找下一个空闲端口（最多到 8799）。Windows 启动器直接报错，需要手动设置环境变量：

```
set HERMES_WEBUI_PORT=8888
```

5.2 文件写入被拒绝 (Edit approval denied)

已在 v0.1 修复。如果还遇到，说明 portable/app/packages/ 里的代码版本旧了，删掉 portable/app/packages/ 重新跑 setup 即可。

5.3 WSL 错误：localhost 代理

说明启动器找到了系统的 WSL 启动器而不是 Git Bash。已在 v0.1 修复。现在 Windows-Start.bat 优先使用 U 盘自带的 PortableGit，不会再踩这个坑。如果还遇到，删掉 portable/app/packages/ 重新启动。

5.4 首次启动下载 Python、Git 很慢

启动脚本检测到国内网络会自动用清华源 + ghfast.top + gh-proxy.com + 华为云镜像。如果还是慢，可以手动下载 standalone Python 和 PortableGit 解压到 portable/app/runtime/ 下。

5.5 想完全重置

```
:: Windows  
rmdir /s /q portable\data\.hermes
```

```
# macOS / Linux  
rm -rf portable/data/.hermes
```

下次启动相当于第一次使用。

6. 给开发者

如果你想改 Hermes 本体或 Web UI 代码：

- **Agent 源码**在 `portable/agent/`，改完后需要重启服务才生效
- **Web UI 后端**在 `portable/webui/`，改完后重启服务
- **Web UI 前端**在 `portable/webui/static/`，改完后刷新浏览器
- 如果改了 `portable/agent/` 的代码，但 `portable/app/packages/` 里有旧副本，需要同步：

```
Copy-Item portable\agent\xxx.py portable\app\packages\xxx.py -Force
```

6.1 仓库 = U 盘骨架

这个仓库本身就是 U 盘的内容（除了大依赖）。git 里的 `portable/` 目录就是 U 盘上的 `portable/`。
`Windows-Start.bat` / `Mac-Start.command` 只是把太大、不适合放 git 的东西（Python 解释器 + PortableGit + pip 包）下载进来。**所以你看到的项目结构 = 用户看到的 U 盘结构。**

本文档使用 Markdown 编写，通过 Edge 浏览器的打印功能转换为 PDF。重新生成请运行 `docs/build_pdf.py`。